Laxmi Charitable Trust's

Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce

Department of Information Technology (Bsc.IT Semester III)

Data Structures Practical

Practical - II

Roll No. :S056	Name:Shweta.Mohan.Singh
Class :SYIT	Batch : B1
Date of Assignment : 10-7-26	Date/Time of Submission :10-7-26

2. Linked List Manipulation:

Write a program to:

a. Create a singly linked list.

INPUT:-

```python
class node:
    def __init__(self,data):
        self.data=data
        self.next=None


node1=node(11)
node2=node(12)
node3=node(14)
node4=node(1)


node1.next=node2
node2.next=node3
node3.next=node4



def traver(head):
    curr=head


    while(curr):
        print(curr.data,end="=>")
        curr=curr.next

    print("Null")


traver(node1)
```

OUTPUT:-

```
=======================
11=>12=>14=>1=>Null
```

b. Insert a node at the beginning, end, and at a given position in a linked list.

INPUT:-

```
class node:
    def __init__(self,data):
        self.data=data
        self.next=None


node1=node(11)
node2=node(12)
node3=node(14)
node4=node(1)


node1.next=node2
node2.next=node3
node3.next=node4


global head
head=node1


def traver(head):
    curr=head


    while(curr):
        print(curr.data,end="=>")
        curr=curr.next

    print("Null")


def insert_beg(data):
    global head
    new=node(data)
    new.next=node1
    head=new

insert_beg(10)
traver(head)
```

OUTPUT:-

```
=========== RESTART: C:/Use
10=>11=>12=>14=>1=>Null
```

B.1

INPUT:-

```
class node:
    def __init__(self,data):
        self.data=data
        self.next=None


node1=node(11)
node2=node(12)
node3=node(14)
node4=node(1)


node1.next=node2
node2.next=node3
node3.next=node4


global head
head=node1


def traver(head):
    curr=head


    while(curr):
        print(curr.data,end="=>")
        curr=curr.next

    print("Null")


def insert_beg(data):
    global head
    new=node(data)
    new.next=node1
    head=new


#Insert at end
def insert_end(data):
    global head
    new=node(data)


    if head is None:
        head=new
    else:
        temp=head
        while temp.next:
            temp=temp.next
        temp.next=new

insert_beg(10)
insert_end(7)
traver(head)
```

OUTPUT:-

```
============ RESTART: C:/Users/itlab/Desktop/Sh
10=>11=>12=>14=>1=>7=>Null
```

B.2

INPUT:-

```
class node:
    def __init__(self,data):
        self.data=data
        self.next=None


node1=node(11)
node2=node(12)
node3=node(14)
node4=node(1)


node1.next=node2
node2.next=node3
node3.next=node4


global head
head=node1




def traver(head):
    curr=head


    while(curr):
        print(curr.data,end="=>")
        curr=curr.next

    print("Null")


def insert_beg(data):
    global head
    new=node(data)
    new.next=node1
    head=new
```

```python
#Insert at end
def insert_end(data):
    global head
    new=node(data)


    if head is None:
        head=new
    else:
        temp=head
        while temp.next:
            temp=temp.next
        temp.next=new

def ins_pos(pos,data):
    global head
    new=node(data)

    temp=head
    for i in range(pos-2):
        temp=temp.next
    new.next=temp.next
    temp.next=new

insert_beg(10)
insert_end(7)
ins_pos(2,17)
traver(head)
```

OUTPUT:-

```
=========== RESTART: C:/Users/itlab/
10=>17=>11=>12=>14=>1=>7=>Null
```

c. Delete a node from a given position in a linked list.

INPUT:-

```python
class node:
    def __init__(self,data):
        self.data=data
        self.next=None


node1=node(11)
node2=node(12)
node3=node(14)
node4=node(1)


node1.next=node2
node2.next=node3
node3.next=node4


global head
head=node1




def traver(head):
    curr=head


    while(curr):
        print(curr.data,end="=>")
        curr=curr.next

    print("Null")
```

```python
def dele_pos(pos):
    global head
    temp=head
    for i in range(pos-2):
        temp=temp.next
    temp.next=temp.next.next

insert_beg(10)
insert_end(7)
ins_pos(2,17)
dele_pos(4)
traver(head)
```

OUTPUT:-

```
=========== RESTART: C:/Users/i
10=>17=>11=>14=>1=>7=>Null
```

Curiosity Questions:

1. Why is inserting a node at the beginning of a singly linked list faster than inserting at the end?
2. What will happen if we don’t update the head pointer when inserting a node at the beginning?
3. Why do we need to use a loop when inserting a node at a specific position in the linked list?
4. What would happen if you forgot to update the next pointer during deletion at a given position?
5. What should you check before deleting a node from a position in the linked list? How would you ensure that your program does not crash with invalid positions or empty list operations?